Agent swarms and the new model economics

Jul 20, 2026

Earlier this year, we ran experiments to test the limits of scaling agents to cooperate toward a goal. Our hypothesis was that this would unlock a new tier of task scale and complexity.

The flagship project was a long-running swarm building a web browser from scratch. It succeeded as a proof of concept, but fell far short of polished software.

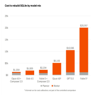
That work was deliberately empirical. We started from a blank canvas and hill-climbed toward a stable, effective system. Since then, our goal has been to understand the agent swarm well enough to engineer it deliberately.

To test that progress, we returned to a task the old swarm had struggled with: building SQLite from scratch, in Rust, from nothing but its documentation.

Our initial results have been promising. We ran the old and new swarms on the same task, with the same models and the same time budget, and measured how much of a held-out SQL test suite each could pass.

The new swarm did better in every model configuration. Using Grok 4.5, it reached 80% in four hours, while the old swarm spiraled and had to be paused before its second hour.

We also varied which models did which jobs. In some runs, one model handled everything while in others, a frontier model planned while a fast, inexpensive model carried out the work. Every mix produced similar quality, but the costs varied enormously.[1]



Trees and leaves

Descriptions of large tasks naturally take the shape of trees, with a goal at the root that subdivides recursively into basic units of work. Our swarm has two roles, both organized around that same tree-like decomposition:

- Planner agents, powered by the smartest models, split a goal into pieces and delegate them.
- Worker agents, generally powered by faster and less expensive models, execute those pieces.

The design is a superset of more rigid orchestration systems. Rather than imposing a fixed topology on the problem, the swarm's shape grows to cover the problem's contours, and compute and context scale in proportion to the task's complexity.

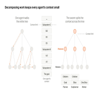
We think this is why the design generalizes to tasks as diverse as building a browser, solving math problems, and optimizing GPU kernels. We've also used it internally to find and fix vulnerabilities in open-source software, raise test coverage on our own codebase, and generate billions of tokens of synthetic training data.

What the tree does for memory

When a single agent takes on a complete task, it has to walk the entire tree itself, descending to each leaf while holding its ancestors, its current position, and the wider goal in context the whole time.

We think this explains why long-running single agents drift. They can either focus on the work in front of them and lose sight of the bigger picture, or hold the big picture and do a worse job on the piece.

In a swarm, a planner never implements, so its context never fills with low-level detail, and a worker never plans, so it can spend all its context on one narrow piece of work.



We suspect the ability to scale the agent swarm comes from this context efficiency, more than from parallelism itself. That efficiency is present in the swarm at every scale, which is why this decomposition helps agent performance even on moderately sized tasks.

There are echoes of this structure elsewhere. The economist Ronald Coase, asking why firms exist at all, argued that coordination costs grow faster than the work itself, so organizations settle into tiers of bounded units rather than letting everyone talk to everyone.

A version control system for agents

In an earlier post about the swarm, we noted that tools like Git and Cargo rely on coarse locks for concurrency control. This is fine for one developer but unworkable for the volume of work produced by hundreds of concurrent agents.

The browser swarm from earlier this year peaked at roughly 1,000 commits per hour on Git. The new system peaks at around 1,000 commits per second.

To facilitate this rate of activity, we built a new version control system (VCS) from scratch. Throughput was not the only reason to own this layer. Every change in the system passes through the VCS, so it is where collisions first become visible, and several of the coordination mechanisms in the next section are implemented directly inside of it.

Failure modes at 1,000 commits per second

Human engineering teams have standard coordination mechanisms like code review, ownership, standups, and merge queues. Those systems work at human tempo, but at the commit-rate of the swarm, we see failure modes that human teams don't routinely encounter.

Split-brain design

Two planners, unaware of each other, implement the same concept in different ways in different parts of the codebase.

We fixed this through prompting. Planners make design decisions themselves rather than delegating them, and we require them to ensure that no two delegated subtrees decide the same question.

Contention between planners

A harder form of contention is when two planners know about each other and fight through back-and-forth changes over the same files.

The problem is two pictures of reality, and merge tooling can't fix a disagreement. Instead, we have agents record decisions in shared design docs. Code that depends on a decision carries a compile-checked reference back to its doc. When planners unknowingly contradict each other, a reconciler merges the docs and the references propagate the resolution downstream.

Merge conflicts

Within the swarm, agents constantly collide on the same files. In order to resolve a collision they would have to stop, absorb the other agent's context, and merge around it. Worker agents are bad at this and, in practice, either overwrite the other change or abandon their own.

To fix this, we created a system where a neutral third-party agent intervenes on merge conflicts and resolves them on behalf of all parties. Its only goal is to be impartial and efficient, similar to the way merge queues work in engineering teams.

Megafiles

Some files are particularly popular places for agents to work. Each agent might add only a small amount of code, and no single agent is responsible for keeping the files small.

These "megafiles" choke everything. They're expensive to transport, diff, and merge, and become the site of constant collisions.

To fix this, we gave worker agents a way to flag bloated files. Once flagged, we block new commits and an outside agent decomposes the overgrown file into smaller modules.

Ossification

Agents have learned, from working in existing codebases with humans in the loop, not to touch core code even when it needs to change.

To fix this, we license intentional breakage. An agent that judges a core change worthwhile can make a focused patch outside its scope and leave a comment explaining why it did it.

The compiler carries the change through the rest of the system, and everything depending on the old design fails to build. Each agent that hits one of those errors finds the comment, reads the reasoning, and updates its own piece of work to match.

Review lenses

In a system that is both long-running and multi-agent, errors accumulate, and the swarm needs a way to correct itself before small mistakes become foundational.

We experimented with many kinds of review lenses, such as giving a review agent the worker's full transcript, or only its output, or nothing but the codebase. We also tried reviewers running on different models, with different training and a different personality.

No single lens catches everything, but decorrelated lenses stack, the way self-driving systems reach above-human reliability without any single perfect component. The compute spent on review is high return, since review is much cheaper than the work it audits. We suspect this stacked review system was a major contributor to the sustained quality of the runs.

Letting agents shape the environment

Stigmergy is the mechanism by which swarm organisms like ants and termites coordinate without direct communication. They shape the environment, and the environment shapes the next organism.

We had encoded rules like "keep notes" and "document decisions" in earlier runs because they seemed obviously good. In retrospect, they were letting agents institutionalize knowledge for their future selves and teammates.

We pushed this further with an experiment in self-authored, shared context we call the Field Guide. It's a folder owned entirely by the agents, whose index.md is automatically injected into every agent at start. It is the agents' job to curate what goes into the guide and their only constraint is a line budget.

The underlying logic of the guide is that model weights are frozen, so it's precisely surprise encounters that are worth capturing so the next agent trajectory is shorter.

The Field Guide is an early experiment with promising results. We'd expect the benefits to be even larger on codebases agents don't fully own. Training models to write for their successors, where better capture leads to better rewards, is an interesting follow-up area of research.

The SQLite experiment

We instructed the new version of the swarm, equipped with all the improvements described above, to implement the whole of the 835-page SQLite manual in Rust. We withheld the source code, test suites, SQLite binary, and internet access.

To measure progress, we graded against sqllogictest, a test suite from the SQLite project built to check that different database engines return the same results for the same queries. It contains millions of queries with known correct answers, and the grade is the fraction the swarm's database gets right. Progress shows up as a rising curve over the course of a run.

The swarm was never told the suite existed. After each run, we manually reviewed the code and the run itself, checking for cheating and shortcuts, and confirming the system was built out evenly, rather than just in the places where the tests look.

As you read the curves, keep in mind that agents chose their own strategies. Some built broad foundations and scored low for hours before a late spike while others went deep on one area, scored early, then plateaued while filling in the rest. Trends matter more than exact scores at exact moments.

Results across model mixes

We tested four configurations spanning capability and cost:

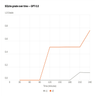
1. **GPT-5.5 as both planner and worker.** A strong frontier model throughout.[2]
2. **Grok 4.5 as both planner and worker.** Our cost-efficient frontier model, as a comparison point.
3. **Opus 4.8 as planner and Composer 2.5 as worker.** Frontier judgment paired with efficient execution.
4. **Fable 5 as planner and Composer 2.5 as worker.** To see whether a next-tier planner makes the hybrid more or less worthwhile.

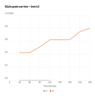
The new harness outperformed the old in every mix.

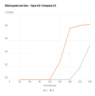
The Fable 5 hybrid passed about two-thirds of the suite within the first hour. By the four-hour cutoff, the new runs sat between 73% and 85%, while the old runs ranged from 11% to 77%.

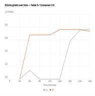
The old Grok 4.5 run was paused before its two-hour mark (more below). Every new configuration went on to pass 100% of the suite.

In the future we'd like to run the full N×N matrix of planner-worker combinations. For this cycle, the comparison that matters is between harness versions, and the behavioral differences turned out to be much larger than the score differences suggest.





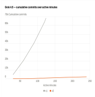




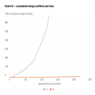
A deep dive into the runs

Starting with the simplest measure of activity, we can see how the rate of commits varied for Grok 4.5 under the old harness versus the new. The old run produced 68,000 commits in its first two hours, roughly 70 times the new run's pace.

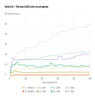
One reading is that it was more productive. Another is that most of those commits were busywork (thrash, contention, churn).



The merge conflict data points to the latter interpretation. The old run accumulated more than 70,000 conflicts before we paused it, accelerating rather than stabilizing, while the new run logged fewer than a thousand over its full four hours.

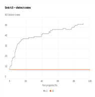


The conflicts concentrated where files grew largest. In the old run, the biggest files kept growing for the entire run and its single hottest file collected 7,771 conflicts, touched by 1,173 different agents. In the new run, the most contested file in the whole codebase saw 47.

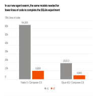


The old swarm's biggest coordination failure — split-brain, or planners duplicating each other's work — showed up in the package structure. Rust code is organized into packages called crates, and in a project like this, each crate is roughly one major component.

The old run sprawled to 54 crates, including three separate SQL packages. The new run settled on nine crates early and never added another.



All of this shows up in the final codebase. In the Fable 5 mix, both the old and new swarms ultimately passed the full suite, but the old one needed 64,305 lines of engine code and the new one did it in 9,908. The Opus mix shows the same shape with 19,013 lines at a 97% grade under the old harness, and 4,645 lines at 100% under the new harness.

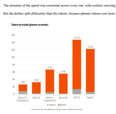


Model economics

We said at the top that every model mix produced similar quality while the costs varied enormously, from $1,339 for the Opus 4.8 hybrid to $10,565 for GPT-5.5 alone. The token data shows where that difference comes from.

The structure of the spend was consistent across every run, with workers carrying at least 69% of the tokens, and over 90% in most.

But the dollars split differently than the tokens, because planner tokens cost more. In the Opus 4.8 and Composer 2.5 mix, the Opus-as-planner produced a small fraction of the tokens but roughly two-thirds of the cost, while Composer-as-worker handled the vast majority of the tokens for the remaining third of the cost.



Few moments in a large task genuinely require frontier intelligence, such as the original decomposition, the design decisions, and certain trade-offs. Once a frontier planner has collapsed the ambiguity into a detailed, explicit instruction, less expensive models simply have to follow it. This is a huge potential source of cost savings. In the run that used GPT-5.5 for both planners and workers, the workers alone cost $9,373. In the run where Opus 4.8 did the planning and Composer 2.5 did the work, the entire worker fleet cost $411.

One detail worth noting comes from comparing the two hybrid runs. The Fable 5 planner ran up a slightly smaller bill than the Opus 4.8 planner, despite roughly twice the per-token price, because it used far fewer planning tokens. But the Fable run's workers went through several times as many tokens, and the run as a whole came out substantially more expensive.

Specs as prompts

Each jump in AI capability has raised the level of abstraction at which an engineer can work.

Autocomplete let engineers work one line of code at a time. Early models raised that to a block of code, and agents raised it to a file or a feature.

With swarms, the unit of work becomes the spec.

For that to work, the swarm has to actually follow the spec, which is what much of this post is about. We gave the swarm 835 pages of prose and it came back with a database. What was scarce in this experiment, and what we expect to be scarce in software engineering going forward, is the right description of intent.

Seen this way, the swarm starts to resemble a compiler. A compiler translates source code down to machine code through a series of intermediate steps. The swarm does something similar with intent. Planners parse a goal into task trees, then lower it step by step into executable work. The difference is that a compiler preserves meaning at every step while the swarm is probabilistic at every one. Everything described in this post exists to close that gap.

We invite you to explore the swarm's output. The codebase from the solo Opus 4.8 run is public at github.com/cursor/minisqlite. Based on our initial glance it looks great, but we have not done a deeper manual analysis. Take your own look, and tell us what you find.